Reasoning and explaining all year round

A mathematical statement has a domain and hypotheses. Before proving it, write them down; to refute it, give an example that satisfies the hypotheses and contradicts the conclusion. A figure or a simulation helps the search but does not prove a universal statement.

Tool	Task and output	Targeted review
Membership and sets	Describe a line as the set of points; check all coordinates.	Application 1, p. 132.
Quantifiers and negation	Negate a universal positivity statement and give a witness index.	R10; application 2.
Implication and conv.	Distinguish fixed point and convergence; give a divergent sequence.	Application 4.
Equivalence	Solve a logarithmic equation after setting the domain.	Application 1.
Proof by induction	Identify the property, initialise and prove the inductive step at an arbitrary index.	Applications 3 then 5.
Counterexample	Remove one hypothesis of the IVT; show what fails.	Application 5.
Contrapositive	Show that two vectors whose dot product is non-zero are not orthogonal.	R60 and R61.
Contradiction	Suppose the four vertices of a non-flat tetrahedron lie in one plane.	Unit 10.2.
Case analysis	Deal with a zero parameter before dividing.	Application 5.
Sufficient condition	Distinguish a size guaranteed by a bound on an exact minimum size.	Applications 4 and 5.

A reusable oral protocol

In one minute: state the object studied and the conclusion, name the tool and its assumptions, give the decisive step, then a check. The partner listens without interrupting and reformulates the conclusion ; a question then asks why an assumption is needed. The "One-minute talk" tasks in each chapter serve as material.

Brief feedback from the listener

Is the object defined? Is the key assumption named? Is the link between the steps understandable? Is the result interpreted? Choose a single point to improve before start again.

Progression of the oral work

September: read a subscript and state the induction hypothesis. November: compare two limits or two counts. December: distinguish existence and uniqueness, then two conditional probabilities. February: describe a projection and continuous change. March: interpret the unit of an integral. April: distinguish probabilistic guarantee from certainty. May: present a complete problem and respond to an objection. These short slots complement workshops O1 to O3.

Results assumed and proofs to reuse

Operations on and composition of limits, monotone convergence, the IVT, the general equivalent characterisations of convexity, the general existence of antiderivatives and the additivity of variance under independence are assumed within the framework given in class. The proofs listed are given in full and are the subject of reproduction or adaptation. A proof of the IVT by bisection, the method of Newton, oblique asymptotes and combinations with repetition are possible extension topics: knowing them is not a hidden prerequisite of the proposed assessments.

Practical-work extension: compare the samples

The program below extends P5-P6. The tolerance in the counting is only there to absorb the decimal representation of floats; the exact calculation uses rational fractions.

```
"""Compare the means of several independent samples."""
from random import Random
from statistics import mean, pstdev
from math import sqrt, comb
from fractions import Fraction

def experience(n, repetitions=1000, p=0.5, graine=2026):
    if n <= 0 or repetitions < 2 or not 0 <= p <= 1:
        raise ValueError("Parameters outside the domain")
    rng = Random(graine)
    moyennes = [sum(rng.random() < p for _ in range(n))/n
                for _ in range(repetitions)]
    return {"n": n, "moyenne": mean(moyennes),
            "ecart_type_observe": pstdev(moyennes),
            "ecart_type_theorique": sqrt(p*(1-p)/n),
            "proportion_ecart_au_moins_0_1":
                sum(abs(m-p) >= 0.1-1e-12 for m in moyennes)/repetitions}

def ecart_exact_equilibre(n, delta):
    """delta is a Fraction: integer boundaries are exact."""
    if n <= 0 or delta <= 0:
        raise ValueError("n and delta must be positive")
    favorables = sum(comb(n,k) for k in range(n+1)
                     if abs(Fraction(k,n)-Fraction(1,2)) >= delta)
    return Fraction(favorables,2**n)

if __name__ == "__main__":
    import json
    print(json.dumps([experience(n) for n in (10,100,1000)],indent=2))
```